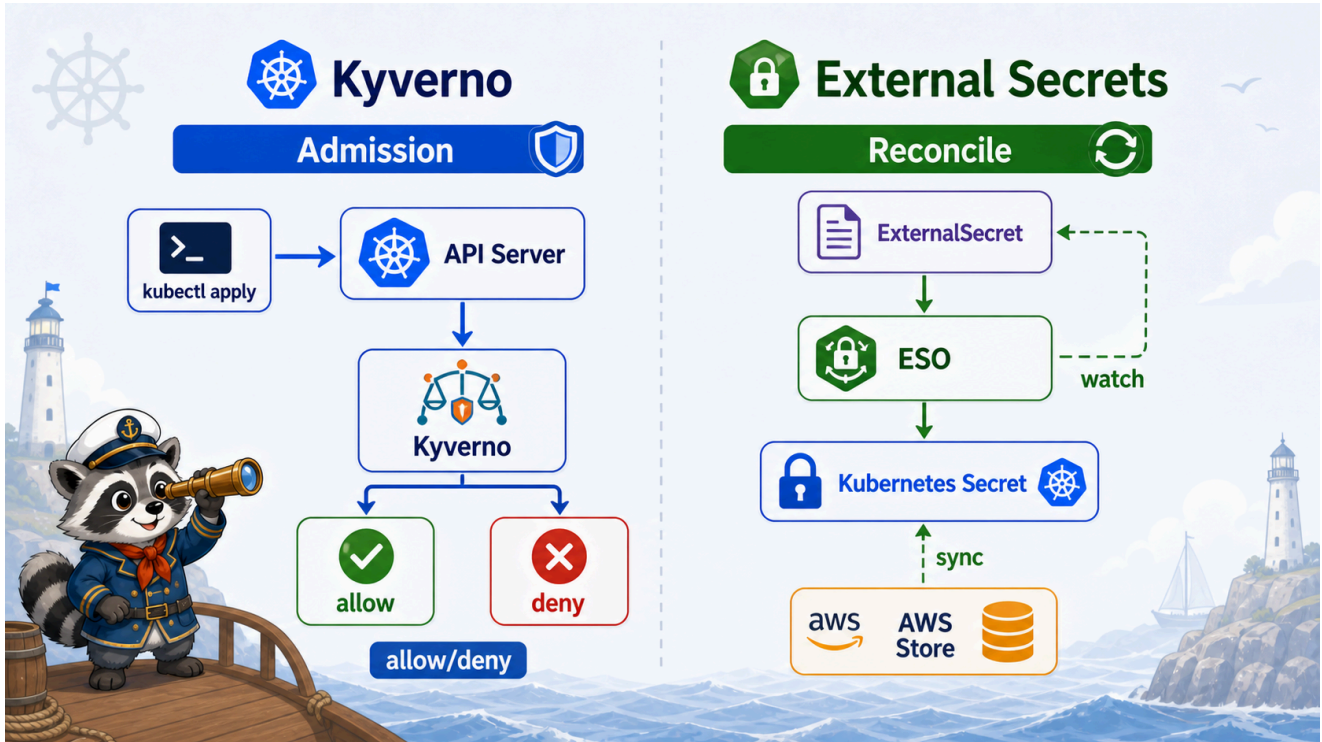


4교시: Kyverno와 External Secrets Operator 설치/Preview



수업 목표

- Kyverno가 admission controller로 동작하는 위치를 이해한다.
- Helm으로 Kyverno를 설치하고 controller Pod, CRD, webhook을 확인한다.
- 설치 실패와 policy 실패를 구분한다.
- External Secrets Operator가 admission controller가 아니라 reconciliation operator라는 점을 구분한다.

Kyverno를 왜 쓰는가

RBAC은 사용자의 권한을 제한한다. 하지만 권한이 있는 사람이 나쁜 manifest를 배포하는 것은 별개의 문제다.

예:

```
image: nginx:latest
securityContext:
  privileged: true
volumes:
```

```
- hostPath:  
  path: /
```

이런 manifest는 권한 있는 사용자가 만들 수도 있다. Kyverno는 admission 단계에서 object 내용을 검사한다.

설치 전 확인

```
kubectl config current-context  
kubectl get nodes  
helm version --short
```

namespace 확인:

```
kubectl get ns kyverno
```

없어도 괜찮다. Helm 설치에서 --create-namespace를 사용한다.

Helm repo

```
helm repo add kyverno https://kyverno.github.io/kyverno/  
helm repo update
```

chart 확인:

```
helm search repo kyverno/kyverno
```

수업에서는 긴 --set 대신 repo에 있는 values file을 사용한다.

```
cat week4/day4/labs/kyverno/values.yaml
```

설치

```
helm upgrade --install kyverno kyverno/kyverno \  
  --namespace kyverno \  
  --create-namespace \  
  -f week4/day4/labs/kyverno/values.yaml
```

예상:

```
NAME: kyverno
NAMESPACE: kyverno
STATUS: deployed
```

실제 검증 예시:

```
Chart version: 3.8.1
Kyverno version: v1.18.1
STATUS: deployed
```

Pod 확인

```
kubectl -n kyverno get pods
```

예상 component:

Pod	역할
admission-controller	admission webhook 요청 처리
background-controller	background scan
cleanup-controller	cleanup policy 처리
reports-controller	policy report 생성

버전에 따라 Pod 이름과 component 구성이 조금 다를 수 있다. 중요한 것은 admission controller가 Running인지다.

실제 검증 예시:

```
kyverno-admission-controller    1/1 Running
kyverno-background-controller   1/1 Running
kyverno-cleanup-controller      1/1 Running
kyverno-reports-controller      1/1 Running
```

CRD 확인

```
kubectl get crd | grep kyverno
```

대표:

```
clusterpolicies.kyverno.io
policies.kyverno.io
policyreports.wgpolicyk8s.io
```

CRD가 없으면 policy manifest를 적용할 수 없다.

webhook 확인

```
kubectl get validatingwebhookconfiguration | grep kyverno
kubectl get mutatingwebhookconfiguration | grep kyverno
```

admission controller는 webhook으로 API Server와 연결된다.

```
API Server
-> Kyverno webhook
-> allow/deny
```

설치가 느릴 때

Kyverno는 webhook과 controller가 준비될 때까지 시간이 걸릴 수 있다.

```
kubectl -n kyverno wait --for=condition=Ready pod --all --timeout=180s
```

Pod가 Ready가 아니면:

```
kubectl -n kyverno describe pod <pod-name>
kubectl -n kyverno logs <pod-name>
```

Helm으로 설치하는 이유

방식	수업 기준
Helm	release/values/uninstall이 명확
remote YAML apply	버전/출처/삭제 기준이 흐려짐
수동 manifest 복붙	반복 재현 어려움

W4D1 이후 add-on 설치하는 Helm으로 통일한다.

설치 후 바로 policy를 넣지 않는 이유

먼저 Kyverno 자체가 건강한지 확인해야 한다.

확인	이유
Pod Running	webhook 처리 가능
CRD 존재	policy resource 생성 가능
webhook 존재	API Server와 연결
Helm release deployed	uninstall/upgrade 가능

설치가 불안정한 상태에서 policy가 실패하면 원인이 policy인지 설치인지 구분하기 어렵다.

Kyverno와 External Secrets Operator의 차이

W4D4에는 Kubernetes 보안 도구가 둘 이상 등장한다. 둘 다 controller처럼 보이지만 위치와 목적이 다르다.

도구	주 역할	동작 위치
Kyverno	manifest를 허용/거부/변형/검사	API Server admission 단계
External Secrets Operator	외부 secret store 값을 Kubernetes Secret으로 동기화	controller reconciliation loop

Kyverno는 잘못된 object가 cluster에 들어오기 전에 막을 수 있다.

```
kubectl apply
-> API Server
-> Kyverno admission webhook
-> allow/deny
```

External Secrets Operator는 이미 생성된 ExternalSecret 같은 custom resource를 watch하다가 원하는 Kubernetes Secret 상태를 만든다.

```
ExternalSecret desired state
-> ESO controller reconcile
-> AWS Secrets Manager/SSM Parameter Store 조회
-> Kubernetes Secret 생성/갱신
```

ESO 설치 preview

실제 설치하는 환경과 cloud credential 준비 상태에 따라 선택한다. 설치한다면 W4 공통 원칙대로 Helm을 사용한다.

```
helm repo add external-secrets https://charts.external-secrets.io
helm repo update

helm upgrade --install external-secrets external-secrets/external-secrets \
  --namespace external-secrets \
  --create-namespace
```

확인:

```
helm list -n external-secrets
kubectl -n external-secrets get deploy,pod
kubectl get crd | grep external-secrets
```

local kind에서는 AWS credential을 붙이지 않기 때문에 실제 Secrets Manager 조회까지 강제하지 않는다. 오늘의 목표는 "operator가 어떤 권한과 API object를 필요로 하는가"를 이해하는 것이다.

ESO object preview

External Secrets Operator는 보통 SecretStore 또는 ClusterSecretStore, 그리고 ExternalSecret 리소스를 사용한다.

SecretStore

-> 어느 외부 provider를 볼 것인가

ExternalSecret

-> 어떤 외부 key를 어떤 Kubernetes Secret key로 동기화할 것인가

AWS provider에서는 Secrets Manager와 SSM Parameter Store를 연결할 수 있다. W5 AWS에서 실제 secret store 선택 기준을 다시 다룬다.

Evidence Note

```
# W4D4S4 Kyverno install
- Helm release:
- Kyverno namespace Pod:
- CRD:
- ValidatingWebhookConfiguration:
- 설치 중 막힌 지점:
- ESO를 설치한다면 확인할 것:
- Kyverno와 ESO 동작 위치 차이:
```

한 줄 요약

Kyverno는 admission 단계에서 배포를 막고, External Secrets Operator는 외부 secret을 Kubernetes Secret으로 동기화하는 reconciliation controller다.